```python
import openai
from langchain import
    ChatOpenAI

def generar_respuesta(prompt):
    modelo = ChatOpenAI(
        model = "gpt-4o"
        temperature = 0.7
    )

    respuesta = modelo.invoke(
    ) prompt
    return respuesta.content
)

if __name__ == "__main__":
    prompt = "Explícame qué
    es la inteligencia artificial
    print(generar_respuesta(prompt)
```

UNIDAD 8:

ENTRENAMIENTO SUPERVISADO

CURSO IA

Docente:
Gabriel Mosso

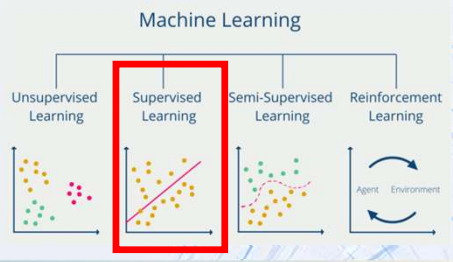
BRAINSTORM
Rep.Argentina 786, Sunchales,Sta fe.

GabrielNMosso@GMail.com
Tel: +54 351 3733383

ENTRENAMIENTO SUPERVISADO

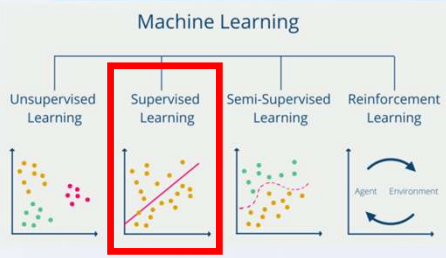
El **entrenamiento supervisado** es una de las formas más comunes y fundamentales de enseñar a una inteligencia artificial.
Se basa en el uso de **datos etiquetados** o sea cada ejemplo que se le muestra al modelo ya viene con la **respuesta correcta**.

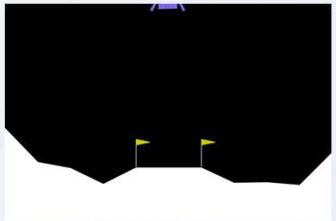
Aspecto	Descripción
Definición	Método de entrenamiento basado en datos etiquetados, donde cada entrada tiene una salida conocida.
Funcionamiento	1. Se entregan ejemplos con entradas y salidas correctas. 2. El modelo aprende a relacionar entradas con salidas. 3. Se evalúa con datos nuevos.
Usos principales	- Clasificación (asignar categorías). - Regresión (predecir valores numéricos).
Ejemplos	- Detectar spam en correos electrónicos. - Predecir el precio de una casa.- Reconocimiento de imágenes.
Requisitos	- Gran cantidad de datos etiquetados. - Etiquetas consistentes y precisas. - Datos representativos del problema.
Ventajas	- Buen rendimiento si los datos son adecuados. - Evaluación objetiva. - Ideal para problemas concretos con resultados esperados.
Desventajas	- Costoso en tiempo y recursos por la necesidad de etiquetado. - Poco flexible con datos no etiquetados. - Riesgo de sobreajuste si no se controla bien.

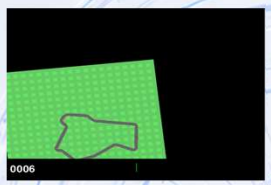


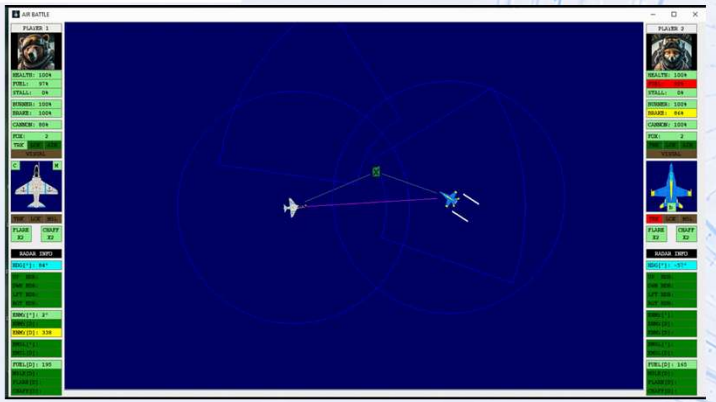
ENTRENAMIENTO SUPERVISADO: GIMNASIOS DISPONIBLES

Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org / https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io / https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, combate, supervivencia.	





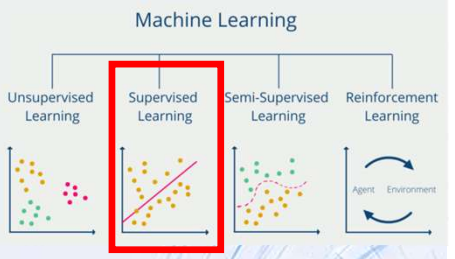


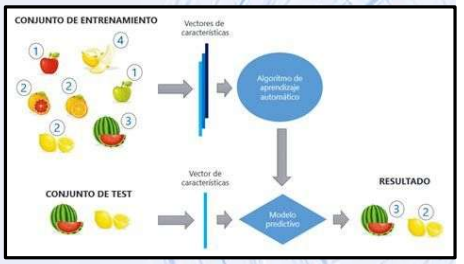


ENTRENAMIENTO SUPERVISADO [IMPLEMENTACION HIBRIDA]

Etapa Inicial	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
1. Crear dataset manual	Generar un dataset inicial con ejemplos correctos de juego	Jugar partidas manualmente y registrar los estados (X) y comandos (Y)	CSV, pandas, grabación de partidas	Registrar casos incorrectos; Datos desbalanceados; pocas partidas para generalizar
2. Entrenar modelo inicial	Que la DNN aprenda a imitar el comportamiento humano	Definir arquitectura de la DNN, compilar, usar .fit() con dataset	TensorFlow/Keras, .fit(), pérdida binary_crossentropy	Overfitting por pocas partidas; aprender patrones erróneos si los datos son malos

Etapa Cíclica	Objetivo	Tareas específicas	Herramientas / Técnicas comunes	Errores frecuentes
3. Ciclo de auto-juego y reentrenamiento	Refinar progresivamente la política del agente	Subdividido en pasos cíclicos		
3.1 Generar partidas automáticas	Que la DNN explore y produzca nuevas experiencias	Ejecutar partidas con el modelo actual; almacenar estados (X) y acciones (Y)	model.predict(), entorno AirBattle, listas o arrays para registrar episodios	Dataset demasiado repetitivo; falta de exploración de nuevas estrategias
3.2 Modificar dataset con reglas	Mejorar dataset aplicando heurísticas de desempeño	Revisar los estados y acciones; aplicar reglas programadas (ej: HEALTH<0.2 → CMD_BRAKE=1)	NumPy, Python, reglas if-else, reward shaping	Aplicar reglas incorrectas; sobre-corregir acciones; sesgo excesivo
3.3 Reentrenar modelo con dataset modificado	Ajustar la política de la DNN basándose en nuevas experiencias	Cargar modelo con load_model y entrenar con .fit() usando el dataset modificado	TensorFlow/Keras, .fit(), model.save/load	No normalizar entradas; entrenar con demasiadas pocas o demasiadas épocas; olvidar guardar modelo
3.4: Evaluación de métricas del modelo	Analizar qué tanto aprendió el modelo y **cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs)**.	detectar **variables inútiles, correlaciones falsas** o **comportamientos inesperados** antes de lanzar otra ronda de autogeneración de partidas.	Analisis de impacto, Pesos Efectivos, Correlación Inputs outputs.	Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos, Interpretación equivocada de pesos, Omitir verificación con lógica del dominio.





MANAGER ENTRENAMIENTO
El objetivo del script unificador es **orquestar todas las etapas** para que el entrenamiento sea cíclico y autónomo:

- **Cargar el modelo inicial o crear uno nuevo.**
- **Generar partidas automáticas** (el modelo juega contra sí mismo).
- **Recolectar datasets** (ganador/perdedor, jugador 1/jugador 2).
- **Modificar y filtrar datasets** aplicando las reglas del dominio.
- **Reentrenar el modelo** con los datos actualizados.
- **Evaluar métricas** (impacto de entradas, correlaciones, pesos).
- **Guardar el modelo actualizado.**

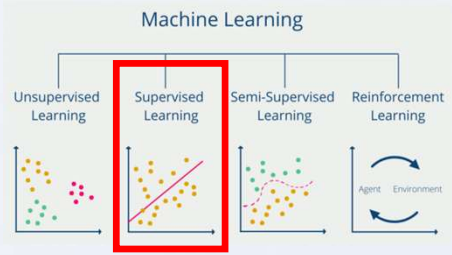
Repetir el ciclo N veces o hasta que se cumplan criterios de mejora.

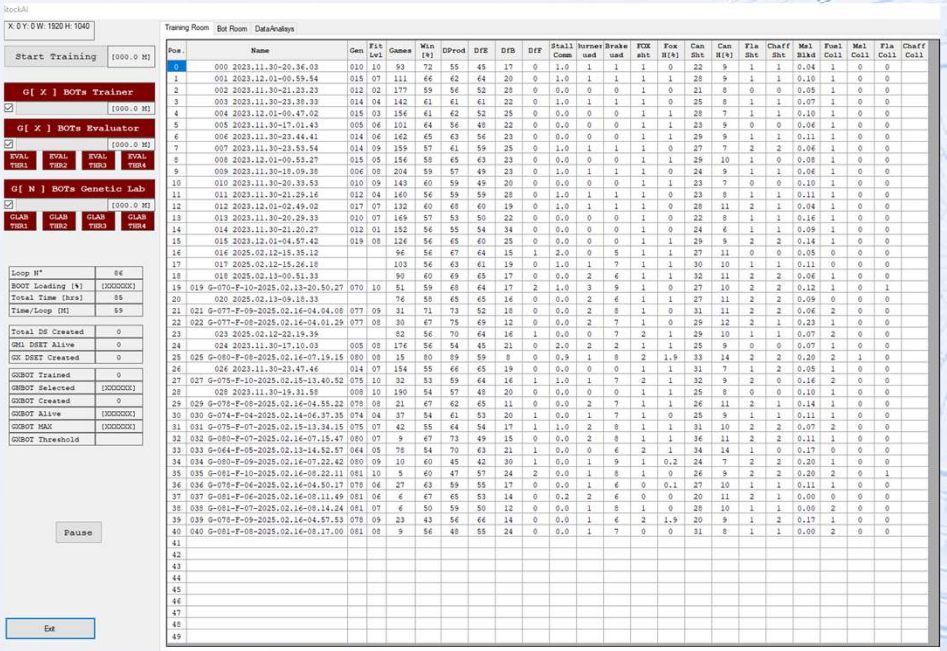
Funciones principales del MANAGER:
El script debería estructurarse en **módulos o funciones**, cada una llamando a los scripts ya existentes:

- generar_partidas() → llama al script que simula partidas y genera datasets.
- modificar_datasets() → procesa datasets y aplica reglas de consistencia.
- entrenar_modelo() → entrena o reentrena con el dataset combinado.
- evaluar_metricas() → ejecuta los scripts de análisis (impacto, correlación, pesos).
- guardar_modelo() → versiona el modelo (ej: model_v2.h5).

El **manager** las ejecuta en orden dentro de un bucle for ciclo in range(N):
Errores frecuentes en la unificación:

- **Rutas mal configuradas:** cada script puede estar guardando datasets en carpetas distintas; el manager debe unificar rutas.
- **Sobrescribir datasets o modelos:** si no se versionan correctamente, se pierden resultados previos.
- **No controlar la convergencia:** repetir ciclos sin analizar si realmente hay mejora puede llevar a sobreentrenamiento o estancamiento.
- **Falta de logs:** sin guardar un historial de métricas por ciclo, es imposible comparar la evolución.
- **Exceso de automatización:** no dejar puntos de control para revisión humana puede hacer que el modelo aprenda errores acumulados.





Pos.	Name	Gen	Fit Lvl	Games	Win [%]	DProd	DFE	DFB	DFF	Stall Comm	burner uad	Brake uad	FOX abt	Fox H[%]	Can Sht	Can H[%]	Fla Sht	Chaff Sht	Mel Blkd	Fuel Coll	Mel Coll	Fla Coll	Chaff Coll
0	000 2023.11.30-20.36.03	010	10	93	72	55	45	17	0	1.0	1	1	1	0	22	9	1	1	0.04	1	0	0	
1	001 2023.12.01-00.59.54	015	07	111	66	62	64	20	0	1.0	1	1	1	1	28	9	1	1	0.10	1	0	0	
2	002 2023.11.30-21.23.23	012	02	177	59	56	52	28	0	0.0	0	0	1	0	21	8	0	0	0.05	1	0	0	
3	003 2023.11.30-23.35.33	014	04	142	61	61	61	22	0	1.0	1	1	1	0	25	8	1	1	0.07	1	0	0	
4	004 2023.12.01-00.47.02	015	03	156	61	62	52	25	0	0.0	0	0	1	1	28	7	1	1	0.10	1	0	0	
5	005 2023.11.30-17.01.43	005	06	101	64	56	48	22	0	0.0	0	0	1	1	23	9	0	0	0.06	1	0	0	
6	006 2023.11.30-23.44.41	014	06	162	65	63	56	23	0	0.0	0	0	1	1	29	9	1	1	0.11	1	0	0	
7	007 2023.11.30-23.53.54	014	09	159	57	61	59	25	0	1.0	1	1	1	0	27	7	2	2	0.06	1	0	0	
8	008 2023.12.01-00.53.27	015	05	156	58	65	63	23	0	0.0	0	0	1	1	29	10	1	0	0.08	1	0	0	
9	009 2023.11.30-18.09.38	006	08	204	59	57	49	23	0	1.0	1	1	1	0	24	9	1	1	0.06	1	0	0	
10	010 2023.11.30-20.33.53	010	09	143	60	59	49	20	0	0.0	0	0	1	1	23	7	0	0	0.10	1	0	0	
11	011 2023.11.30-21.29.16	012	04	160	56	59	59	28	0	1.0	1	1	1	0	23	8	1	1	0.11	1	0	0	
12	012 2023.12.01-02.49.02	017	07	132	60	68	60	19	0	1.0	1	1	1	0	28	11	2	1	0.04	1	0	0	
13	013 2023.11.30-20.29.33	010	07	169	57	53	50	22	0	0.0	0	0	1	0	22	8	1	1	0.16	1	0	0	
14	014 2023.11.30-21.20.27	012	01	152	56	55	54	34	0	0.0	0	0	1	0	24	6	1	1	0.09	1	0	0	
15	015 2023.12.01-04.57.42	019	08	126	56	65	60	25	0	0.0	0	0	1	1	29	9	2	2	0.14	1	0	0	
16	016 2025.02.12-15.35.12			96	56	67	64	15	1	2.0	0	5	1	1	27	11	0	0	0.05	0	0	0	
17	017 2025.02.12-15.26.18			103	56	63	61	19	0	1.0	1	7	1	1	30	10	1	1	0.11	0	0	0	
18	018 2025.02.13-00.51.33			90	60	69	65	17	0	0.0	2	6	1	1	32	11	2	2	0.06	1	0	0	
19	019 G-070-F-10-2025.02.13-20.50.27	070	10	51	59	68	64	17	2	1.0	3	9	1	0	27	10	2	2	0.12	1	0	1	
20	020 2025.02.13-09.18.33			76	58	65	65	16	0	0.0	2	6	1	1	27	11	2	2	0.09	0	0	0	
21	021 G-077-F-09-2025.02.16-04.04.08	077	09	31	71	73	52	18	0	0.0	2	8	1	0	31	11	2	2	0.06	2	0	0	
22	022 G-077-F-08-2025.02.16-04.01.29	077	08	30	67	75	69	12	0	0.0	2	7	1	0	29	12	2	1	0.23	1	0	0	
23	023 2025.02.12-22.19.39			82	56	70	64	16	1	0.0	0	7	2	1	29	10	1	1	0.07	2	0	0	
24	024 2023.11.30-17.10.03	005	08	176	56	54	45	21	0	2.0	2	2	1	1	25	9	0	0	0.07	1	0	0	
25	025 G-080-F-08-2025.02.16-07.19.15	080	08	15	60	89	59	8	0	0.9	1	8	2	1.9	33	14	2	2	0.20	2	1	0	
26	026 2023.11.30-23.47.46	014	07	154	55	66	65	19	0	0.0	0	0	1	1	31	7	1	2	0.05	1	0	0	
27	027 G-075-F-10-2025.02.15-13.40.52	075	10	32	53	59	64	16	1	1.0	1	7	2	1	32	9	2	0	0.16	2	0	0	
28	028 2023.11.30-19.31.58	008	10	190	54	57	48	20	0	0.0	0	0	1	1	25	8	0	0	0.10	1	0	0	
29	029 G-078-F-08-2025.02.16-04.55.22	078	08	21	67	62	65	11	0	0.0	2	7	1	1	26	11	2	1	0.14	1	0	0	
30	030 G-074-F-04-2025.02.14-06.37.35	074	04	37	54	61	53	20	1	0.0	1	7	1	0	25	9	1	1	0.11	1	0	0	
31	031 G-075-F-07-2025.02.15-13.34.15	075	07	42	55	64	54	17	1	1.0	2	8	1	1	31	10	2	2	0.07	2	0	0	
32	032 G-080-F-07-2025.02.16-07.15.47	080	07	9	67	73	49	15	0	0.0	2	8	1	1	36	11	2	2	0.11	1	0	0	
33	033 G-064-F-05-2025.02.13-14.52.57	064	05	78	54	70	63	21	1	0.0	0	6	2	1	34	14	1	0	0.17	0	0	0	
34	034 G-080-F-09-2025.02.16-07.22.42	080	09	10	60	45	42	30	1	0.0	1	9	1	0.2	24	7	2	2	0.20	1	0	0	
35	035 G-081-F-10-2025.02.16-08.22.11	081	10	5	60	47	57	24	2	0.0	1	8	1	0	26	9	2	2	0.20	2	0	1	
36	036 G-078-F-06-2025.02.16-04.50.17	078	06	27	63	59	55	17	0	0.0	1	6	0	0.1	27	10	1	1	0.11	1	0	0	
37	037 G-081-F-06-2025.02.16-08.11.49	081	06	6	67	65	53	14	0	0.2	2	6	0	0	20	11	2	1	0.00	0	0	0	
38	038 G-081-F-07-2025.02.16-08.14.24	081	07	6	50	59	50	12	0	0.0	1	8	1	0	28	10	1	1	0.00	2	0	0	
39	039 G-078-F-09-2025.02.16-04.57.53	078	09	23	43	56	66	14	0	0.0	1	6	2	1.9	20	9	1	2	0.17	1	0	0	
40	040 G-081-F-08-2025.02.16-08.17.00	081	08	9	56	48	55	24	0	0.0	1	7	0	0	31	8	1	1	0.00	2	0	0	

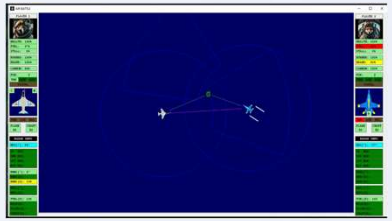
Etapa 1: Crear Dataset Manual

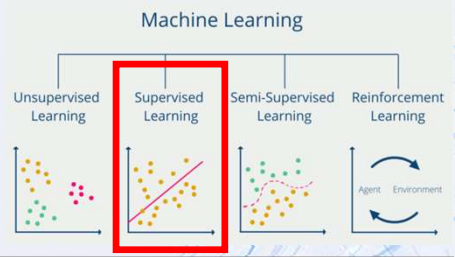
Generar un dataset inicial que contenga ejemplos correctos de juego, es decir, pares **estado → acción** donde la acción tomada sea razonablemente buena. Este dataset servirá como base para entrenar la red neuronal antes de que aprenda a jugar por sí misma.

Tareas específicas:
- Jugar partidas manualmente en AirBattle.
- Asegurarse de que los datos cubran distintos escenarios: atacar, esquivar, recolectar ítems, etc.
- Crear DataSet unificado.

Herramientas / Técnicas comunes:
- CSV o archivos Excel para almacenar datos.
- Pandas para manipular y limpiar los datasets.
- Scripts en Python para convertir logs del juego en tablas de estado y comandos.

Errores frecuentes:
- **Pocas partidas para generalizar:** El dataset solo refleja tus decisiones en un tipo de escenario, la red no aprende situaciones diversas.
- **Sesgo del jugador:** Si siempre juegas agresivo o pasivo, la DNN puede aprender una estrategia limitada.
- **Duplicación de datos:** Filas repetidas que sobre-representan ciertos estados y distorsionan el aprendizaje.
- **Falta de exploración de casos extremos:** No incluir situaciones críticas (salud baja, enemigos cercanos, pocos recursos) que son importantes para que la DNN aprenda a reaccionar.
- **Falta de diversidad de escenarios:** Si todas las partidas ganadas se obtienen con la misma estrategia, la DNN no aprenderá a adaptarse a situaciones distintas.





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Tareas específicas:

Definir la arquitectura de la DNN:

Entradas: 40 sensores del avión.
Capas ocultas: p.ej., 2 capas densas con 10 neuronas cada una.
Salidas: 8 comandos (CMD_LEFT, CMD_RIGHT, ...).

Preparar los datos:

Leer los CSV de ganador y perdedor.
Normalizar las entradas (HEALTH, FUEL, etc.) para que los valores estén en rangos compatibles con la función de activación (sigmoid).
Opcional: ponderar los ejemplos de ganadores más alto que los perdedores para reforzar buenas acciones.

Compilar el modelo:

Seleccionar **optimizador** (adam, sgd).
Seleccionar **función de pérdida** adecuada (binary_crossentropy) porque las salidas son acciones binarias.

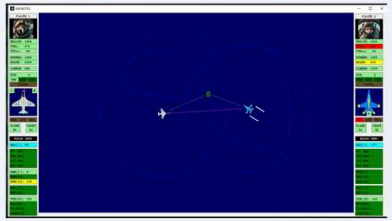
Entrenar con .fit():

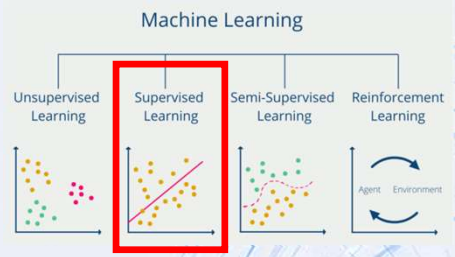
Ajustar el modelo usando el dataset manual.
Definir número de épocas, tamaño de batch y validación si es necesario.

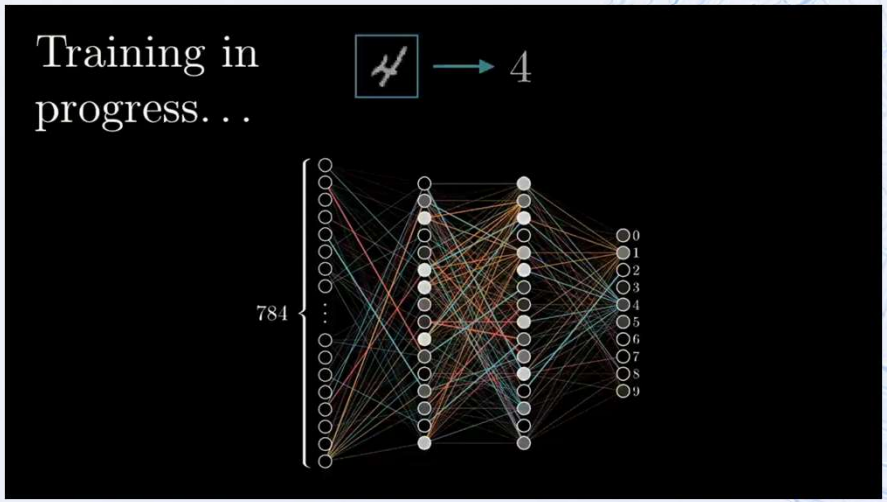
Guardar el modelo entrenado:

Para poder cargarlo en etapas posteriores y continuar el entrenamiento iterativo.

model.save("airbattle_model_initial.keras")







SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"

Etapa 2: Entrenar modelo inicial

Que la DNN aprenda a **imitar las decisiones correctas de las partidas manuales**, utilizando los datasets generados por AirBattle (ganador y perdedor). Esta etapa establece la **política inicial** del agente antes de que comience a jugar contra sí mismo y a refinarse.

Errores frecuentes

Overfitting por pocas partidas:
La DNN memoriza patrones específicos de las partidas manuales y no generaliza bien a nuevas situaciones.

Dataset desequilibrado:
Si hay muchos más ejemplos de perdedor que de ganador, la DNN puede aprender estrategias incorrectas.

No normalizar las entradas:
Valores como HEALTH o FUEL en escalas muy diferentes pueden hacer que la DNN aprenda lentamente o mal.

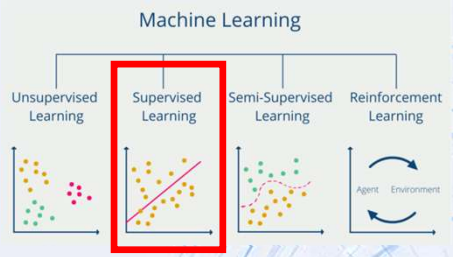
Arquitectura inadecuada:
Muy pocas neuronas/capas → incapaz de aprender patrones complejos.
Muy grande → entrenamiento lento y riesgo de overfitting.

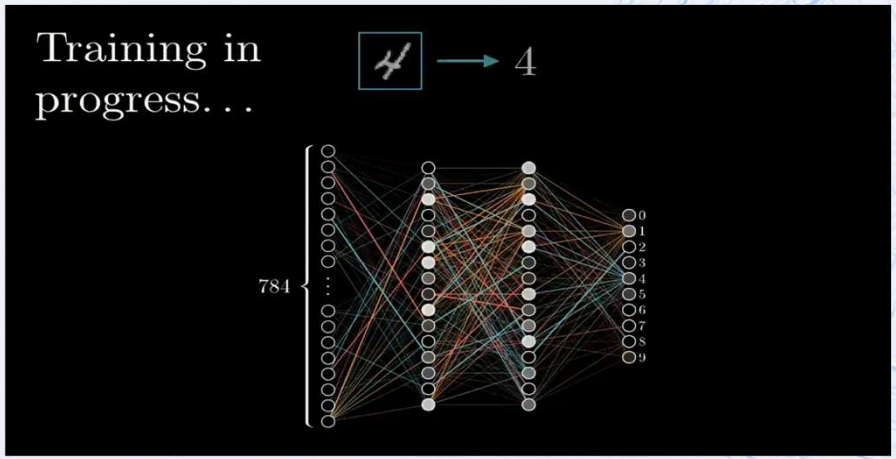
Funciones de activación incorrectas:
Usar relu en la capa de salida para acciones binarias puede producir resultados no binarios; se recomienda sigmoid o softmax según el enfoque.

Batch o número de épocas inapropiado:
Demasiadas épocas → sobreajuste; pocas épocas → aprendizaje incompleto.
Tamaño de batch demasiado grande → poca variabilidad por actualización; demasiado pequeño → entrenamiento ruidoso.

No validar durante entrenamiento:
Sin un dataset de validación, no se detecta si el modelo está aprendiendo correctamente o simplemente memorizando ejemplos.

Ignorar secuencia temporal:
.fit() trata cada fila como independiente; si el juego tiene secuencia de estados, la DNN podría no capturar ciertas dinámicas temporales (esto se corrige más adelante con reglas o RL).





SCRIPT >>> " TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"

Etapa 3.1: Generar partidas automáticas

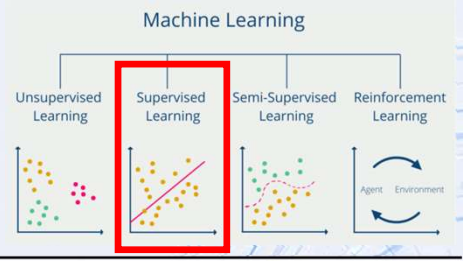
Permitir que la DNN juegue contra sí misma para **producir nuevas experiencias** que luego se usarán para mejorar la política del agente. Esto permite que la red aprenda a enfrentarse a situaciones más diversas que las observadas en las partidas manuales.

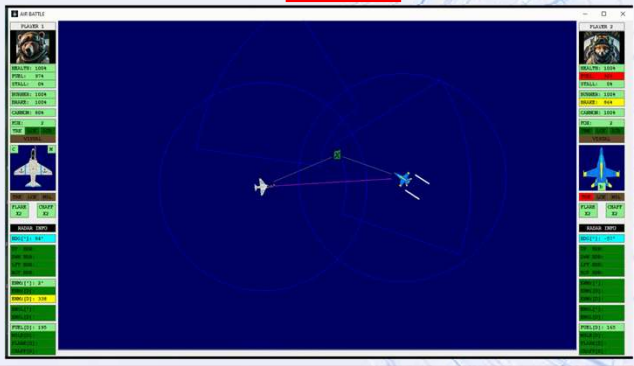
Tareas específicas:

- **Cargar el modelo entrenado inicialmente:** Se usa el modelo entrenado en la Etapa 2.
- **Configurar el entorno AirBattle:**
  - Inicializar la partida para que ambos jugadores sean controlados por la DNN.
  - Registrar cada frame los estados (X) y las acciones tomadas (Y).
- **Ejecutar la partida automáticamente:**
  - En cada paso del juego, predecir las acciones usando model.predict(state).
  - Aplicar un umbral (p.ej., >0.5) para decidir si activar cada comando.
  - Guardar estados y comandos en arrays o listas.
- **Al finalizar la partida:** AirBattle genera dos datasets automáticamente: uno para el **ganador** y otro para el **perdedor**.

Errores frecuentes

- **Dataset demasiado repetitivo:** Si la DNN juega siempre igual, los estados y acciones generados no aportan información nueva.
- **Falta de exploración:** La DNN puede quedarse atrapada en estrategias subóptimas ("política local"), porque solo replica lo aprendido.
- **Desalineación de estados y acciones:** Guardar un estado con la acción de un frame diferente puede corromper el dataset.
- **Errores de sincronización con el juego:** Latencia o retrasos en la predicción pueden generar acciones aplicadas tarde o inconsistentes.
- **Sobrepeso de un jugador:** Si uno de los jugadores DNN es consistentemente más fuerte, el dataset del perdedor podría ser demasiado amplio y no reflejar situaciones útiles.
- **Fallos en la predicción de la DNN:** Salidas continuas sin binarizar correctamente los comandos pueden generar datos incorrectos.
- **Archivos corruptos o no guardados:** Partida interrumpida por error del juego o fallo de disco puede provocar datasets incompletos.
- **Ignorar diversidad de escenarios:** Si siempre se inicializa la partida de la misma manera, los datos generados no cubren todas las posibles situaciones del juego.





SCRIPT >>> "TRAIN01_EJECUTADOR_AIR_BATTLE.py"

SCRIPT >>> "COMMS01_KB_PLAYER.py"

SCRIPT >>> "COMMS02_AI_PLAYER.py"

Etapa 3.2: OPTIMIZACION del DATASET

Objetivo:
Refinar el dataset generado por las partidas automáticas para **corregir errores, eliminar datos irrelevantes y aplicar reglas heurísticas** que mejoren la política de la DNN antes del reentrenamiento.

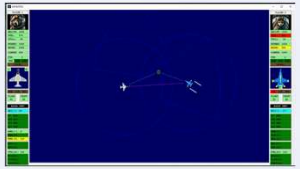
Tareas específicas:
Unificar datasets de partidas ganadoras:
- Recorrer la carpeta de archivos .Battle generados por AirBattle.
- Concatenar todos los archivos en un único dataset unificado (Dataset_Unificado.txt).

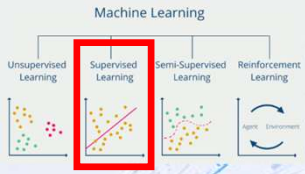
Depuración de datos:
- Leer el dataset unificado y **balancear** las diferentes situaciones para que cada una tenga un **proporción** representativa.
- Contar estadísticas de filas totales, descartadas y conservadas.

Aplicar reglas heurísticas sobre los datos:
- **Ejemplo 1:** Si STALL > 0.7, desactivar comandos de giro (CMD_LEFT y CMD_RIGHT) para evitar perder control del avión.

- **Ejemplo 2:** (ya incluido) filas con todos los comandos en cero se descartan automáticamente.
  Estas reglas **mejoran la calidad del dataset**, asegurando que la DNN no aprenda comportamientos peligrosos o inútiles.

Errores frecuentes en esta etapa
- **Reglas demasiado estrictas:** Aplicar correcciones excesivas puede eliminar variabilidad útil y limitar la capacidad de la DNN de aprender estrategias alternativas.
- **Descartar filas útiles:** Filas donde todos los comandos son cero podrían representar situaciones de espera estratégica; descartarlas siempre puede eliminar casos válidos.





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Un dataset de partidas crudas nunca está listo para entrenar una DNN.
Contiene:

- Ruido humano
- Filas sin información
- Atajos espurios
- Desequilibrios entre situaciones frecuentes y raras.

La Etapa 3 —**Dataset Engineering**— es donde se transforma ese material crudo en un dataset apto para *Behavior Cloning*, aplicando un conjunto organizado de técnicas que la comunidad de Imitation Learning usa de forma estándar.

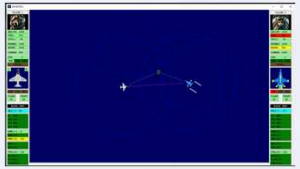
Antes de entrar en las técnicas, una distinción conceptual que atraviesa toda la etapa:

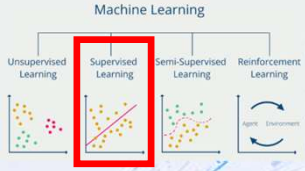
- **Regla:** Una condición puntual y aislada (si X entonces Y).
- **Política:** Un conjunto organizado de reglas que persigue un objetivo coherente (seguridad, navegación, combate, recuperación...).

- **SIT (Situación):** ¿La condición física ocurrió? (ej.: hay una amenaza dentro de rango)
- **COR (Corrección):** Dado que ocurrió, ¿hizo falta además modificar el comando para resolverla?

Con esto en mente, la Etapa 3 se organiza en las siguientes presentaciones:

- Ingestion y Dataset Cleaning
- Policy Engineering (inyección de conocimiento experto)
- Anti-Leakage Sanitization
- Dataset Filtering y Dataset Balancing
- Label Engineering
- Action Masking
- Dataset Randomization y Dataset Validation





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Ingestion and Normalization

Objetivo
Convertir múltiples partidas registradas por el simulador en un único dataset consistente, y dejar los datos en un estado "neutro" antes de aplicar ningún criterio experto.

Tareas específicas

Unificación de partidas:
Recorrer la carpeta de partidas ganadoras generadas por el simulador.
Concatenar todas en un único dataset maestro, respetando un esquema de columnas fijo (no el de cada archivo individual, que puede variar levemente entre versiones del simulador).

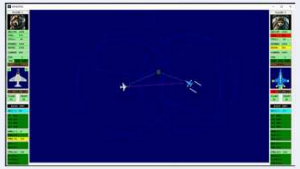
Normalización / Feature Scaling:
Cada sensor físico tiene su propia escala natural (energía, combustible, ángulos, distancias). Antes de que una DNN pueda aprender de ellos hace falta llevarlos a un rango común.

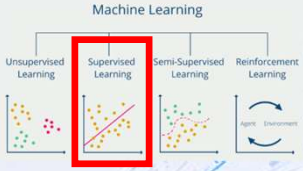
Los sensores angulares (que oscilan entre un valor negativo y uno positivo simétrico) no se escalan igual que los sensores de magnitud pura (que van de 0 a un máximo**): tratarlos con la misma fórmula genérica introduce sesgos.**

Errores frecuentes en esta etapa:

Confundir Cleaning con Filtering: cleaning prepara la fila, no decide si se queda o se va.

Aplicar la misma fórmula de normalización a sensores angulares y de magnitud: produce compresiones o expansiones de rango incorrectas.





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Policy Engineering (Inyección de Conocimiento Experto)

Objetivo
Reemplazar el comando crudo del humano por una etiqueta generada por reglas expertas organizadas en políticas, de forma que la DNN aprenda una política *consistente* y no los errores o inconsistencias de un jugador humano cansado, distraído o simplemente subóptimo.

Concepto clave: jerarquía de políticas
En cualquier instante, más de una política podría "querer" tomar el control de la dirección del avión (evadir una amenaza, perseguir un objetivo, buscar un recurso...). Como el avión solo puede girar en una dirección a la vez, hace falta un

mecanismo de arbitraje: una política de prioridad más alta gana y bloquea a las de menor prioridad para ese ciclo. Diseñar bien este orden de prioridades es, en sí mismo, una decisión de diseño tan importante como las reglas individuales.

Tareas específicas
Definir, para cada familia, la condición de
- **SIT** (cuándo se considera que la situación está presente)
- **COR** (cuándo, dada la situación, hace falta modificar el comando).

Implementar el arbitraje de prioridad de forma explícita (una bandera o mecanismo equivalente que impida que una política de menor prioridad sobrescriba a una de mayor prioridad ya aplicada en ese ciclo).
Documentar el orden de prioridades elegido y su justificación táctica.

Familia	Qué resuelve	Prioridad típica
Políticas de Seguridad	Amenazas que comprometen la supervivencia inmediata (proyectiles entrantes, límites del área de juego)	Máxima
Políticas de Recuperación	Estados físicos degradados del propio agente (pérdida de control, sobrecalentamiento, etc.) que deben corregirse sin importar qué más esté pasando	Muy alta
Políticas Tácticas	Uso de armamento u ofensiva (disparo, bloqueo de blancos)	Media-alta
Políticas Defensivas	Contramedidas ante una amenaza detectada (señuelos, interferencia)	Media-alta
Políticas de Navegación	Persecución u orientación hacia un objetivo (enemigo, punto de interés)	Media
Políticas de Gestión de Recursos	Recolección de ítems o recursos del entorno	Baja

Errores frecuentes en esta etapa:

Reglas demasiado estrictas: sobre-corregir elimina variabilidad útil y le quita a la DNN la posibilidad de aprender estrategias alternativas válidas.

- **Ausencia de arbitraje de prioridades**: si dos políticas escriben el mismo comando sin un orden definido, la última que se ejecuta "gana" por accidente, no por diseño.
- **No medir SIT/COR por separado**: sin esta distinción es imposible saber después si una política está sobre-representada porque la situación es común, o porque se está corrigiendo de más.

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Anti-Leakage Sanitization

Objetivo: Detectar y neutralizar sensores que funcionan como un "espejo" del comando que se quiere predecir, para que la red aprenda la relación real estado → acción y no un atajo trivial.

Concepto clave: Causal Confusion

- Es un proceso de higiene de datos que se aplica *después* de que las políticas ya resolvieron los comandos.
- En la literatura de Imitation Learning se lo conoce como mitigar la **Causal Confusion** (De Haan et al., 2019): cuando un sensor está prácticamente correlacionado 1 a 1 con el comando que se quiere predecir, la red aprende a copiarlo en vez de aprender la causa real de la acción.
- Esa "muleta" desaparece —o directamente falla— apenas el modelo tiene que volar solo, sin ese comando previo disponible como atajo.
- Es una de las causas más comunes de que un modelo funcione perfecto en entrenamiento y colapse en inferencia.

Tipos de fuga a buscar

- **Sensores de retorno directo**: variables que en la práctica replican el comando del paso anterior. Aportan cero información causal nueva y deberían neutralizarse sistemáticamente.
- **Sensores acoplados por construcción física**: variables que crecen o decrecen *mientras* se mantiene presionado cierto comando, generando una correlación espuria fuerte aunque no sean un espejo exacto. Acá la solución no siempre es "apagar siempre": conviene preguntarse si, por debajo de cierto umbral de relevancia, el sensor es solo ruido correlacionado (se neutraliza), pero por encima de ese umbral pasa a ser una señal de seguridad real (se preserva, e incluso puede justificar forzar un override de seguridad sobre el comando).

Tareas específicas

- Auditar el esquema de sensores en busca de candidatos a fuga (buscar correlaciones altas entre sensor y comando correspondiente).
- Clasificar cada candidato: ¿es una fuga pura (se neutraliza siempre) o una fuga condicional (se neutraliza solo por debajo de un umbral de relevancia física)?
- Registrar estadísticas de cuántas veces se aplicó cada neutralización/override, para poder auditarlas.

Errores frecuentes en esta etapa

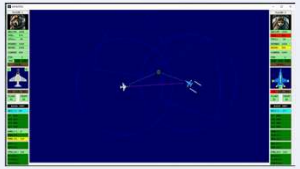
- **No distinguir fuga pura de fuga condicional**: apagar un sensor que en niveles extremos es información de seguridad real elimina señal valiosa, no solo ruido.
- **Aplicar la sanitización antes de que las políticas terminen de resolver los comandos**: el orden importa, porque la sanitización necesita el comando ya definitivo para decidir si corresponde un override.
- **Ignorar esta etapa por completo**: es la causa número uno de que un modelo entrenado con altísima métrica offline se comporte de forma errática al volar de forma autónoma.

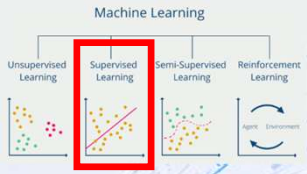
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Dataset Filtering y Dataset Balancing

Objetivo
Asegurar que el dataset final contenga solo filas con información de aprendizaje real, y que ninguna política domine el entrenamiento simplemente por ser más frecuente que las demás.





4.1 Dataset Filtering:

- **Qué se descarta**: filas donde ninguna política terminó activando ningún comando (el agente "no hizo nada" desde el punto de vista de la política). Estas filas no le enseñan nada nuevo a la red sobre la relación estado → acción.

- **Qué se reporta**: cantidad de filas totales, descartadas y conservadas, para poder auditar qué porcentaje del dataset original terminó siendo útil.

Errores frecuentes:

- **Descartar filas útiles**: una fila con todos los comandos en cero podría representar una situación legítima de "espera estratégica" (por ejemplo, mantener rumbo porque nada amerita corrección). Descartarlas siempre, sin matices, puede eliminar casos válidos que la red debería poder reproducir.

- **Filtrar antes de que las políticas terminen de escribir los comandos**: si se filtra sobre el dataset crudo en vez del dataset con políticas ya aplicadas, se descartan filas por el criterio equivocado.

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Dataset Filtering y Dataset Balancing

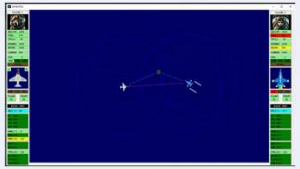
4.2 Dataset Balancing:
Las políticas no ocurren con la misma frecuencia: navegación y persecución son comunes; evasión de amenazas o recolección de ítems raros son escasas. Sin balanceo, la red tiende a converger hacia la política dominante y a ignorar las políticas minoritarias, que suelen ser justamente las más críticas (seguridad).

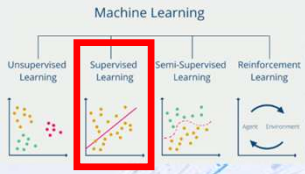
Técnicas estándar a considerar:

- **Oversampling**: duplicar o repetir (con o sin variaciones) las filas de las políticas minoritarias.
- **Undersampling**: reducir la cantidad de filas de la política dominante para emparejar la proporción.
- **Reweighting**: en vez de tocar el dataset, ajustar el peso de cada fila en la función de pérdida según la frecuencia de su política.
- **Balance temporal**: evitar que partidas muy largas (con muchas filas de navegación repetitiva) pesen desproporcionadamente frente a partidas cortas pero ricas en situaciones variadas.

Errores frecuentes:

- **Balancear antes de filtrar**: si se balancea sobre datos que todavía incluyen filas vacías, se termina sobre-representando ruido.
- **Oversampling ingenuo sin variación**: repetir filas idénticas sin ningún tipo de perturbación puede llevar a que la red memorice en vez de generalizar.
- **Balancear por política ignorando la superposición**: una misma fila puede pertenecer a más de una política simultáneamente (por el mecanismo de prioridades); balancear ingenuamente por conteo de "comando activo" puede distorsionar la proporción real de situaciones.





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Label Engineering

Objetivo
Reemplazar etiquetas "duras" (0 o 1 exactos) por representaciones más informativas del nivel de certeza o consenso detrás de cada acción, mejorando la capacidad de generalización de la red.

Label Smoothing
En vez de entrenar con una etiqueta completamente dura (acción correcta = 1, el resto = 0), se introduce incertidumbre intencional: la acción correcta recibe un valor alto pero no absoluto (por ejemplo 0.9), y la probabilidad restante se distribuye entre las demás acciones posibles.

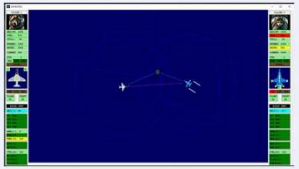
- **Motivación**: evita que la red se sobreajuste a la idea de que "hay una única acción correcta en cada situación", cuando en la realidad muchas situaciones admiten más de una respuesta razonable.
- **Efecto secundario deseable**: mejora la robustez frente a ruido en las etiquetas (por ejemplo, inconsistencias del jugador humano que las políticas no lograron limpiar del todo).

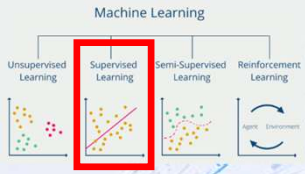
Soft Labels / Confidence Labels
En vez de un único valor fijo de suavizado, la confianza asignada a la etiqueta puede variar según qué tan "clara" fue la situación (una política que se activó por un margen amplio merece más confianza que una que se activó justo en el límite de su umbral).

Teacher Labels
Cuando existe un modelo previamente entrenado (o una política más sofisticada) que puede "opinar" sobre qué acción tomaría en cada estado, sus salidas pueden usarse como etiquetas blandas adicionales (en el espíritu de *knowledge distillation*), en vez de depender únicamente de la política heurística escrita a mano.

Errores frecuentes en esta etapa

- **Suavizar todas las políticas por igual**: una política de seguridad (evasión) no debería suavizarse con el mismo factor que una política de baja prioridad (recolección de ítems); el costo de un error no es el mismo.
- **Confundir Label Smoothing con Balancing**: uno actúa sobre el *valor* de la etiqueta, el otro sobre la *cantidad* de filas. Son técnicas complementarias, no sustitutas.





SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Action Masking

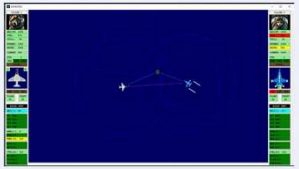
Objetivo
Restringir, en tiempo de inferencia, el espacio de acciones válidas según el estado actual, para que el agente no explore ni sea evaluado sobre acciones físicamente imposibles o inútiles en ese instante.

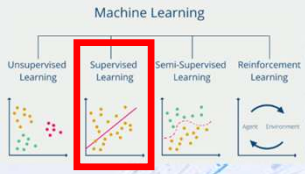
Concepto clave
Action Masking inhibe o fuerza a probabilidad cero la elección de acciones que no tienen sentido dado el estado presente (por ejemplo, una acción ofensiva que depende de un recurso agotado, o una acción de movimiento que llevaría al agente fuera de los límites del entorno).

Nota importante de diseño: a diferencia de las técnicas anteriores, Action Masking no pertenece estrictamente a la etapa de construcción del dataset de entrenamiento — pertenece a la etapa de **inferencia/generación**, es decir, al momento en que el agente ya entrenado está decidiendo qué hacer. Vale la pena entenderlo en esta etapa del curso porque comparte el mismo espíritu que Anti-Leakage Sanitization y Policy Engineering: inyectar conocimiento del dominio para reducir el espacio de hipótesis que la red tiene que aprender por su cuenta.

Ventajas
- Reduce el espacio de búsqueda efectivo.
- Acelera el entrenamiento y la convergencia.
- Evita que el agente "aprenda" a intentar acciones imposibles y sea penalizado por algo que nunca debería haber podido elegir.

Errores frecuentes
- **Implementarlo dentro del pipeline de dataset en vez de en la etapa de inferencia**: mezcla responsabilidades y complica el mantenimiento del código.
- **Enmascarar de forma demasiado agresiva**: si se enmascaran acciones que en realidad sí eran viables en ciertos estados límite, se le quita al agente margen de maniobra legítimo.



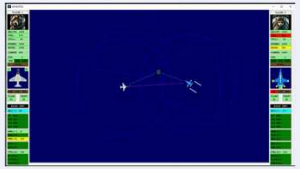


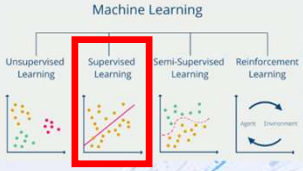
SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.2: OPTIMIZACION del DATASET

Dataset Randomization y Dataset Validation

Objetivo
Cerrar el pipeline asegurando que el orden de las filas no introduzca sesgos de aprendizaje, y que existan métricas objetivas para decidir si el dataset está listo para entrenamiento.





7.1 Dataset Randomization
- Las partidas están, por naturaleza, ordenadas cronológicamente. Si ese orden se preserva, la red puede terminar aprendiendo correlaciones temporales espurias (qué viene "después de qué") en vez de la relación real estado → acción.
- La técnica estándar es mezclar (shuffle) las filas una vez que todas las etapas anteriores ya se aplicaron, de forma que cada fila se evalúe de forma independiente durante el entrenamiento.

7.2 Dataset Validation
Antes de dar por cerrado el pipeline, conviene reportar un conjunto de métricas de salud del dataset:
- Proporción de filas conservadas vs. descartadas en cada etapa.
- Frecuencia de activación de cada política (idealmente expresada como SIT y COR por separado, como se definió en la Presentación 2).
- Cantidad de overrides aplicados por Anti-Leakage Sanitization.
- Cobertura de las políticas de menor prioridad: ¿aparecen con suficiente frecuencia como para que la red tenga ejemplos representativos, o son tan raras que probablemente necesiten Balancing?
- Este reporte final funciona como un "gate" de calidad: si alguna política crítica de seguridad tiene muy pocos ejemplos, es una señal de que hace falta volver a alguna etapa anterior (más partidas, más balanceo, revisión de umbrales) antes de pasar a entrenamiento.

SCRIPT >>> "TRAIN02_UNIFICADOR_DATASET.py"

Etapa 3.3: Reentrenar modelo con dataset optimizado.

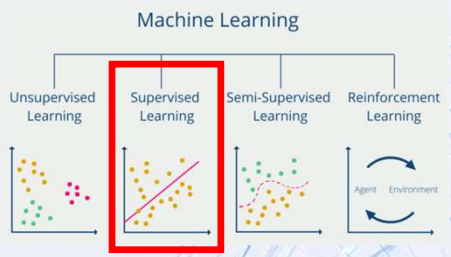
Usar el **dataset depurado y mejorado con reglas heurísticas** para volver a entrenar el modelo. El propósito es que la red neuronal aproveche **tanto la experiencia de las partidas automáticas como las correcciones impuestas por reglas humanas** para alcanzar un comportamiento más robusto y seguro en el entorno AirBattle.

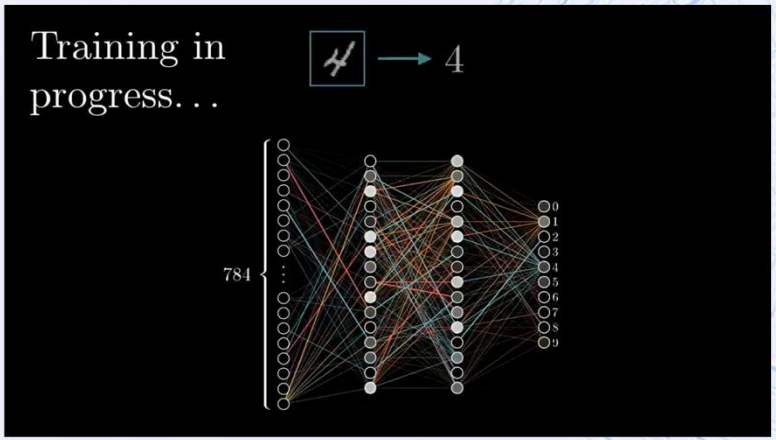
Tareas específicas:

- **Cargar el nuevo dataset (Dataset_Depurado.txt):**
- **Reinicializar o cargar el modelo previo:** Cargar pesos anteriores y **seguir entrenando** para no perder el conocimiento acumulado.
- **Entrenar el modelo con el dataset modificado:** Ajustar batch_size, epochs y optimizador. Monitorear pérdida (loss) y métricas de precisión.
- **Guardar el modelo actualizado:** Exportar pesos y arquitectura en un nuevo archivo (modelo_reentrenado.h5).

Errores frecuentes en esta etapa

- **Sobrescribir el modelo previo sin respaldo:** Se pierde la comparación entre versiones (modelo inicial vs. modelo reentrenado).
- **Overfitting al dataset modificado:** El modelo memoriza las correcciones pero pierde generalización, especialmente si el dataset es pequeño.
- **Reglas mal aplicadas en 3.2 reflejadas en el modelo:** Si las reglas fueron demasiado estrictas o erróneas, el modelo aprenderá un comportamiento sesgado.
- **Olvidar probar en simulación real:** El modelo puede verse “perfecto” en métricas, pero en el juego real tomar decisiones inútiles.





SCRIPT >>> "TRAIN03_AIRBATTLE_ENTRENAMIENTO.py"

Etapa 3.4: Evaluación de métricas del modelo

Analizar qué tanto aprendió el modelo y **cuáles entradas (inputs) tienen más impacto sobre las decisiones (outputs)**.
Esto permite detectar **variables inútiles, correlaciones falsas** o **comportamientos inesperados** antes de lanzar otra ronda de autogeneración de partidas.

Tareas específicas:

Medir impacto de las entradas (6-ANALISIS IMPACTO.py):

- Se extraen los **pesos de la primera capa** de la red neuronal.
- Permite ver **qué inputs (ej. FUEL, STALL, ENEMY_RADAR_TRACK)** son más determinantes en la propagación hacia adelante.
- **Error típico**: confundir magnitud de pesos con importancia absoluta (a veces inputs normalizados cambian la interpretación).

Analizar pesos efectivos Input–Output (8-PESOS EFECTIVOS INPUTS-OUTPUTS.py):

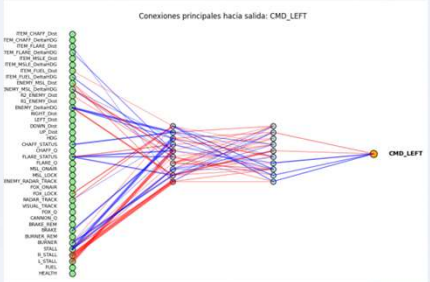
- Se multiplican los pesos de todas las capas para aproximar el "camino" de influencia entre cada input y cada output.
- Permite generar una **matriz de relevancia Input → Output**.
- **Error típico:** interpretar correlaciones espurias como "lógica aprendida".

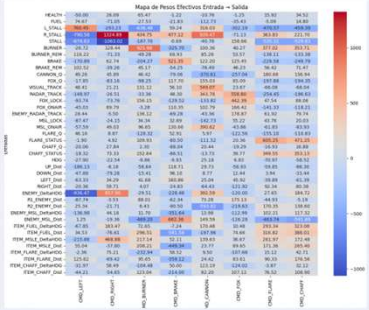
Calcular correlación estadística Input–Output (7-CORRELACION INPUTS-OUTPUTS.py):

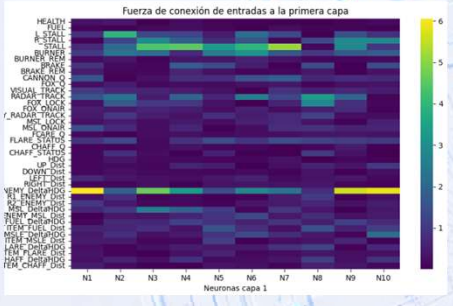
- Se miden coeficientes de correlación (Pearson o Spearman) entre las columnas de entradas y las salidas predichas.
- Diferencia con los pesos: aquí no se mide la red, sino la **relación estadística en los datos y predicciones**.
- **Error típico**: usar correlación lineal en fenómenos no lineales → se pierden relaciones útiles.

Errores frecuentes en esta etapa

Confundir métricas de entrenamiento (loss, accuracy) con análisis de pesos: Una buena accuracy no significa que el modelo esté usando los inputs correctos.
Interpretación equivocada de pesos: Pesos grandes no siempre significan más importancia (puede depender de la normalización de datos).
Omitir verificación con lógica del dominio: Se puede **aprender patrones espurios o irrelevantes** que están en los datos, aunque no tengan sentido desde el punto de vista del sistema real.
No comparar con versiones anteriores: La gracia de esta etapa es ver cómo cambió el modelo respecto al inicial y al reentrenado.







SCRIPT >>> "CHECK01_PESOS_EFECTIVOS_IO.py"

SCRIPT >>> "CHECK02_CORRELACION_IO.py"

SCRIPT >>> "CHECK03_GRAFICO_CONEXIONES.py"

SCRIPT >>> "CHECK04_ANALISIS_IMPACTO_INPUTS.py"

SCRIPT >>> "CHECK05_CAMINOS_EFECTIVOS.py"

SCRIPT >>> "CHECK06_CAMINOS_EFECTIVOS_IMPORT.py"

```
> print("Gracias !")
> print("dudas, consultas...")
> input("Presione una tecla para salir... ")
```


Docente:
Gabriel Mosso

BRAINSTORM
Rep.Argentina 766, Sunchales,Sta fe.

GabrielNMosso@GMail.com
Tel: +54 351 3733383